

Redes Neuronales Artificiales

De la neurona al perceptrón multicapa: activaciones, softmax, backpropagation, entrenamiento e hiperparámetros

¿Qué veremos hoy?

1. **Estructura** de una red y la **neurona**
2. **Funciones de activación** y sus efectos
3. **Softmax** para clasificación multiclase
4. **Funciones de coste** (loss)
5. **Entrenamiento**: backpropagation y descenso de gradiente
6. **MLP**: el aproximador universal
7. **Preprocesado** de los datos
8. **Hiperparámetros** clave
9. **ANN en scikit-learn** y efectos de los hiperparámetros
10. **Optimización**: GridSearchCV → Optuna

 A lo largo de la clase resolveremos **ejercicios numéricos** de activaciones, softmax, función de coste y backpropagation.

 El MLP **no suele ser el mejor modelo para datos tabulares** (ahí gana el *gradient boosting*), pero es la **base** de CNN, RNN, Transformers y GNN. Esa es la razón de estudiarlo a fondo.

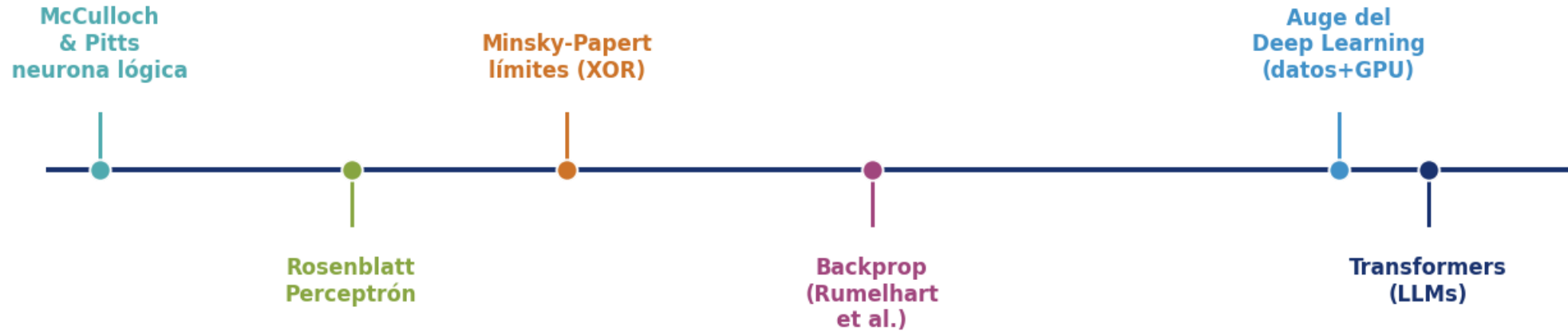


0. Orígenes

De la neurona biológica al perceptrón

Una breve historia

Una breve historia de las redes neuronales



Datos

Hoy abundan → las ANN ganan en problemas grandes.

Cómputo (GPU)

Entrenar redes grandes en tiempo razonable.

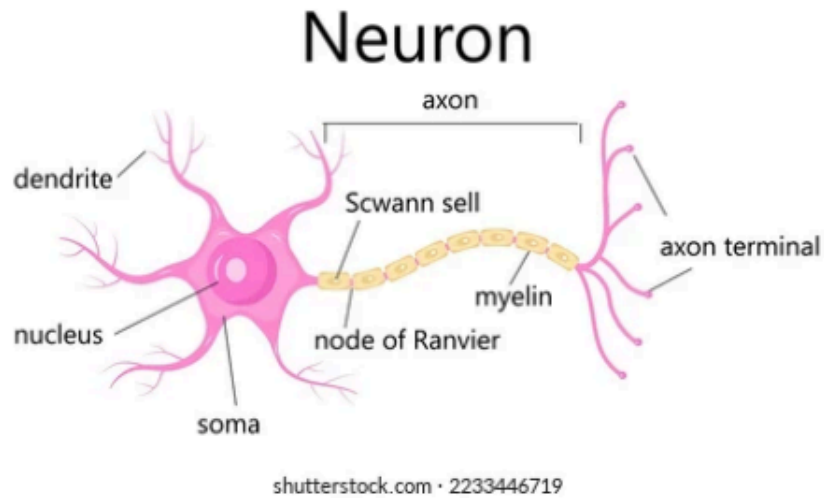
Algoritmos

Mejoras sobre los métodos de los 90.



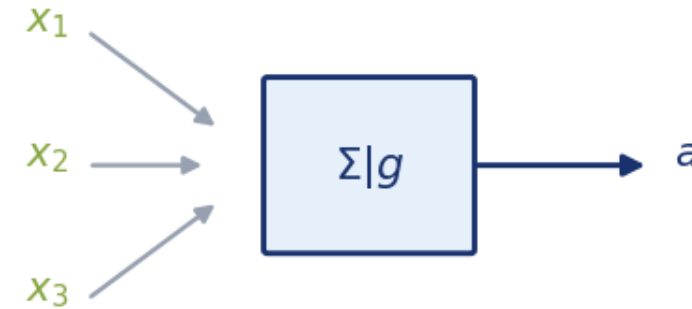
De la neurona biológica a la artificial

Neurona biológica



inspira

Neurona artificial



dendritas → entradas · soma → suma+activación · axón → salida

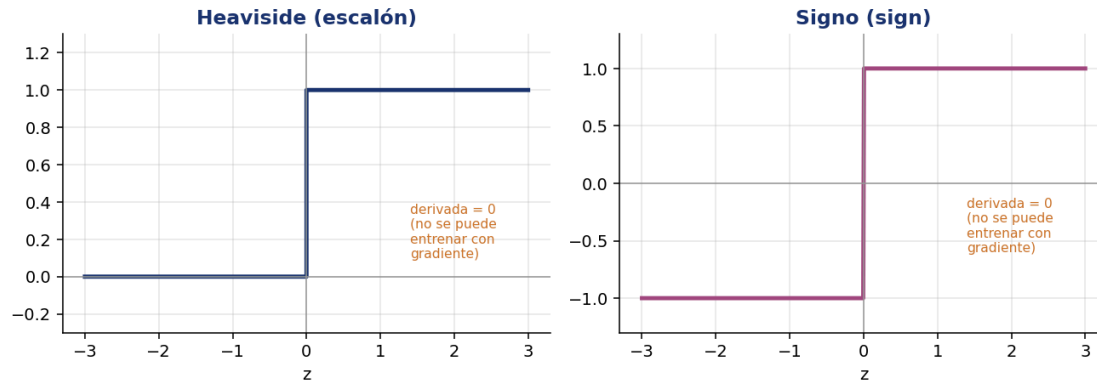
dendritas → **entradas** · soma → **suma + activación** · axón → **salida**

Esquema inspirado en la neurona biológica. Fuente: elaboración propia (inspirado en Géron, 2023, cap. 9).



El perceptrón y las funciones escalón

El perceptrón clásico usa funciones escalón → sin gradiente útil



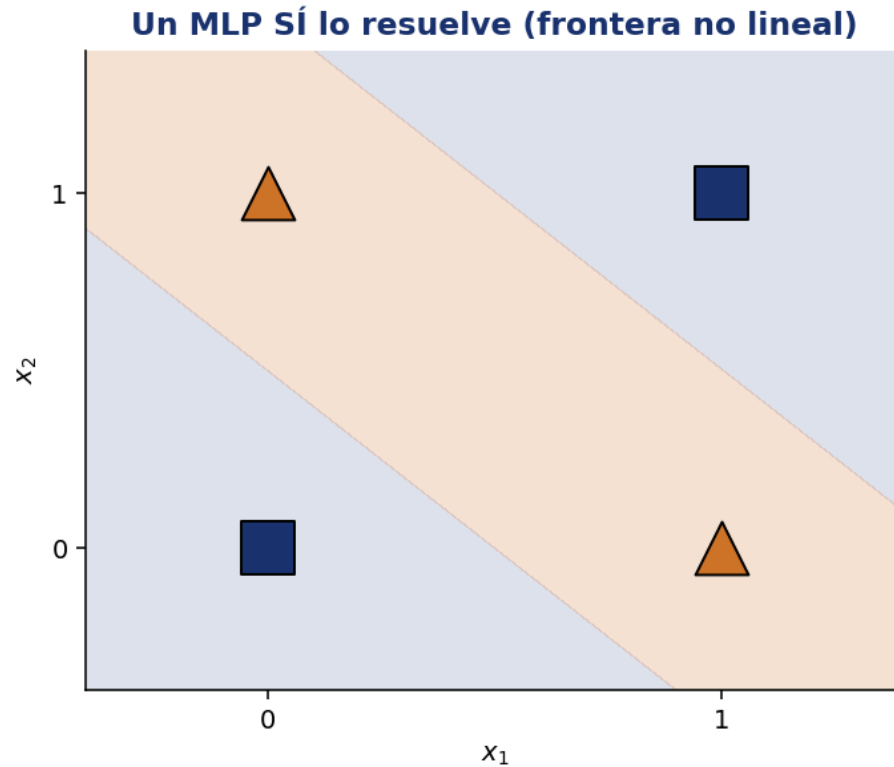
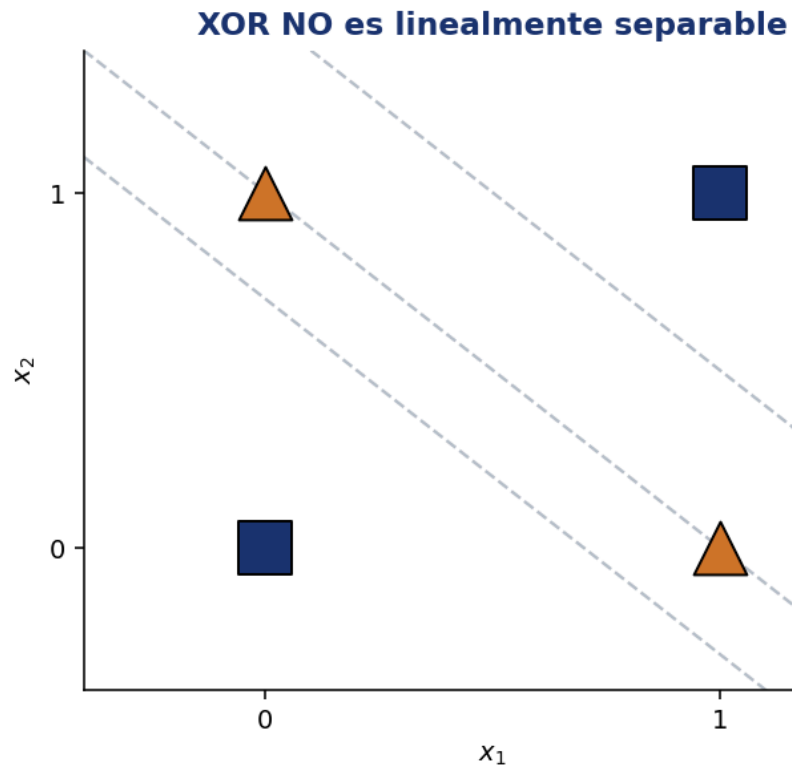
El **perceptrón** (Rosenblatt, 1957) usa una **función escalón**.

Problema: su derivada es **0** → no se puede entrenar con descenso de gradiente.

Reemplazarla por funciones **suaves** (sigmoide...) fue clave para entrenar MLPs.



El problema XOR



Un perceptrón (frontera **lineal**) **no** puede resolver el XOR (Minsky-Papert, 1969). La solución: **apilar capas + no linealidad** → el **MLP**. Esa es la motivación de todo lo que sigue.



1. Estructura de la red

Capas, neuronas y pesos

¿Qué es una red neuronal?

- Una **composición de operaciones matemáticas** organizadas en **capas** de **neuronas** conectadas por **pesos**.
- Cada neurona hace algo simple: **suma ponderada + función de activación**.

Redes simples (MLP)

Arquitecturas modestas, sin GPU.
→ **scikit-learn**, H2O.

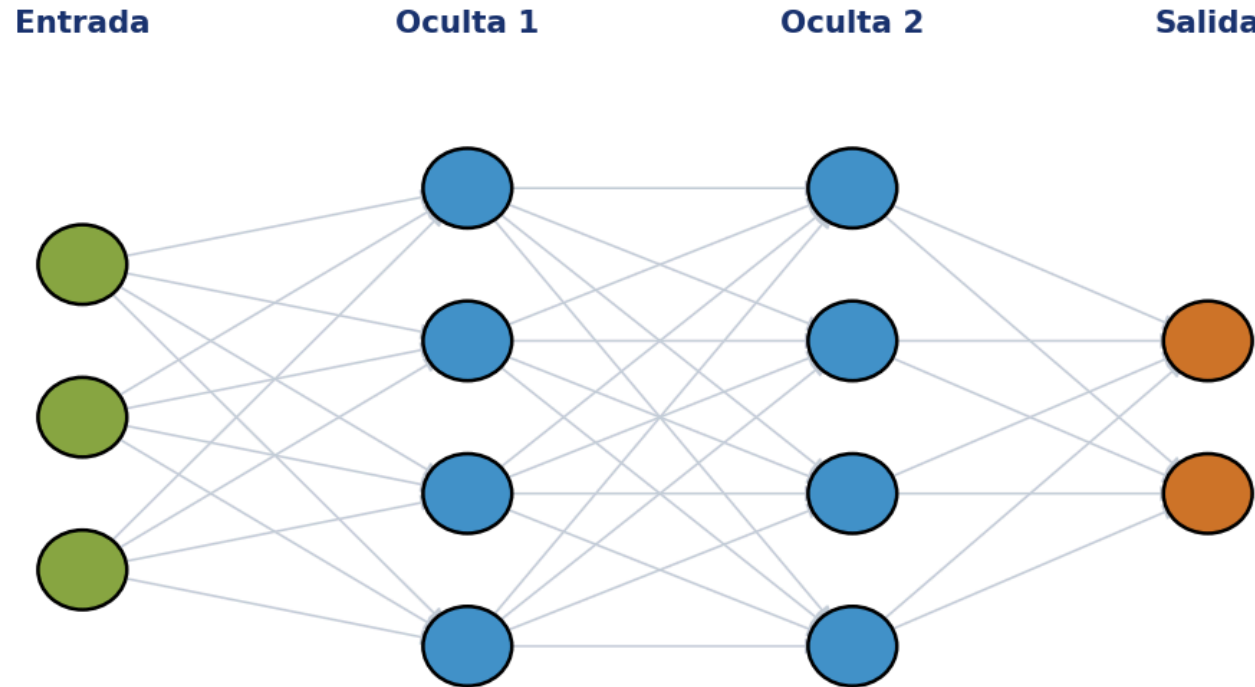
Deep learning

Convolucionales, recurrentes, *transformers*... requieren típicamente GPU.
→ **TensorFlow/Keras, PyTorch**.



Esquema típico de una red neuronal

Perceptron multicapa (MLP): capas densas totalmente conectadas



MLP con dos capas ocultas, totalmente conectado. Fuente: elaboración propia (notes.md).



Anatomía de una red

- **Capa de entrada:** una neurona por variable independiente/predictor.
- **Capas ocultas:** procesamiento no lineal.
- **Capa de salida:** la predicción.
 - **Regresión** → 1 neurona.
 - **Clasificación** → 1 neurona por clase.

Los pesos **regulan** la información que pasa de una neurona a la siguiente. Aprender = **ajustar pesos y bias**.



Una red simple = regresión lineal

Sin capas ocultas y con **activación identidad**:

$$\hat{y} = w_1x_1 + \dots + w_dx_d + b$$

Equivale a una regresión lineal.

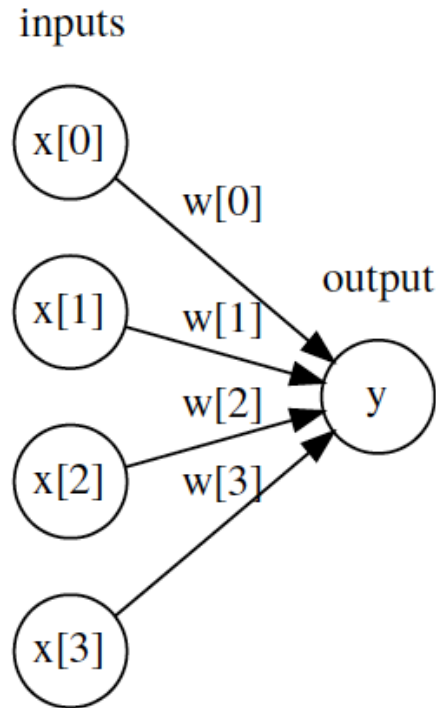
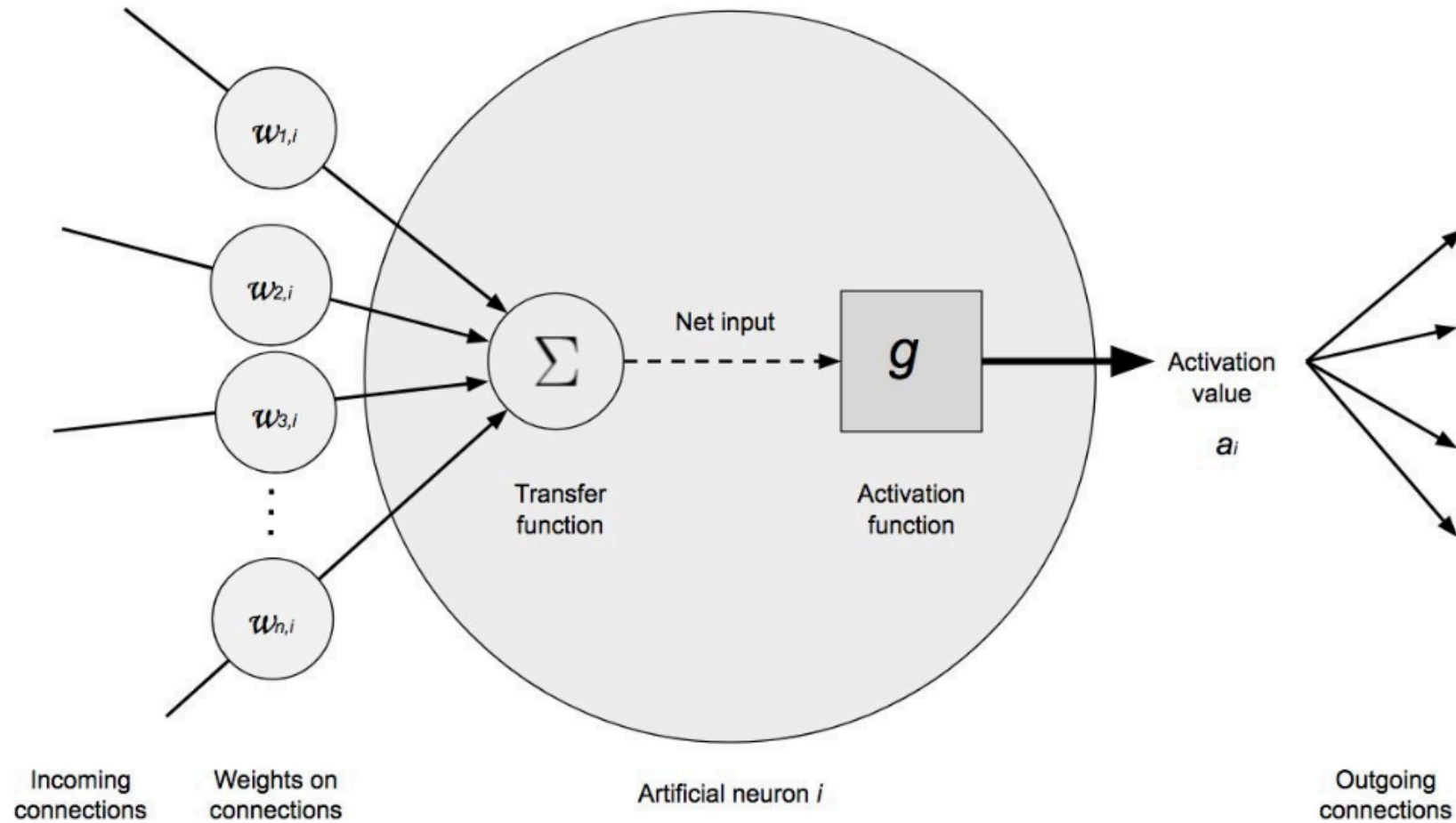


Diagrama: red sin capas ocultas equivalente a un modelo lineal. Fuente: elaboración propia (notes.md).



La neurona (unidad)



Esquema de una neurona: suma ponderada + función de activación. Fuente: elaboración propia (notes.md).



La neurona: matemática

1. Valor neto de entrada (siempre lineal):

$$z = \sum_{i=1}^n x_i w_i + b = \mathbf{x}^\top \mathbf{w} + b$$

2. Función de activación g :

$$a = g(z) = g(\mathbf{x}^\top \mathbf{w} + b)$$

🔑 La **función de activación** es donde reside el poder de la red: sin ella, apilar capas da siempre una **función lineal** de la entrada.



2. Funciones de activación

La fuente de la no linealidad

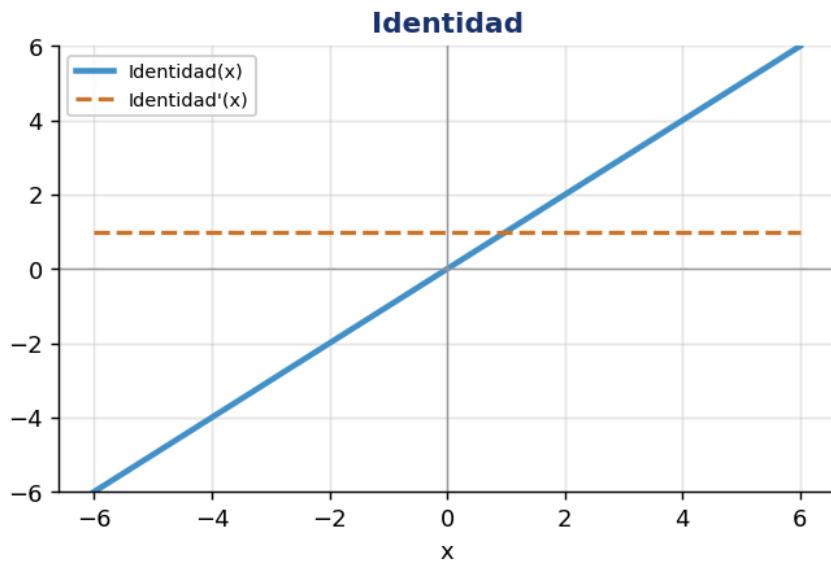
¿Por qué necesitamos no linealidad?

Si **todas** las capas usaran la activación **identidad**, la composición de transformaciones lineales sigue siendo **lineal**. La red entera colapsaría a una regresión lineal, sin importar cuántas capas tenga.

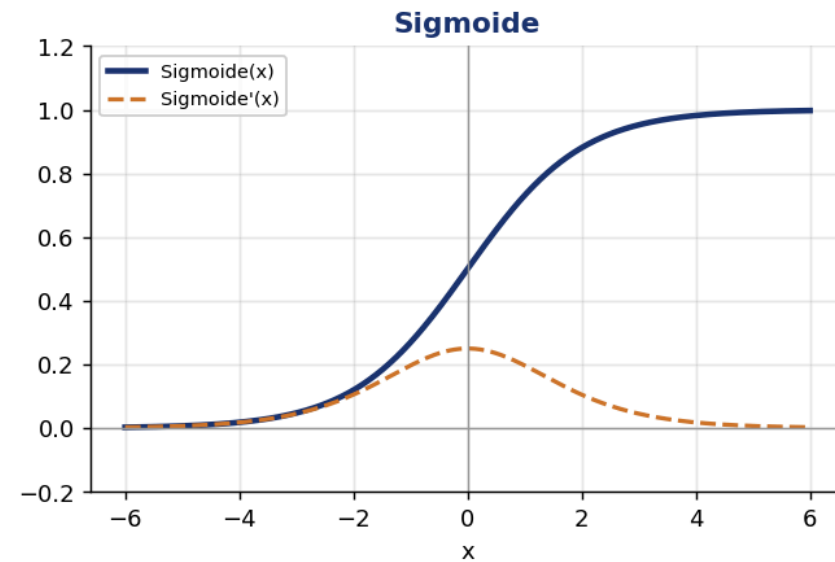
- La **no linealidad** permite aprender fronteras curvas y relaciones complejas.
- También controla, vía su **derivada**, cuánto gradiente fluye hacia atrás (lo veremos en backpropagation).



Identidad y Sigmoide



$g(z)=z$, $g'(z)=1$. Salida en regresión.



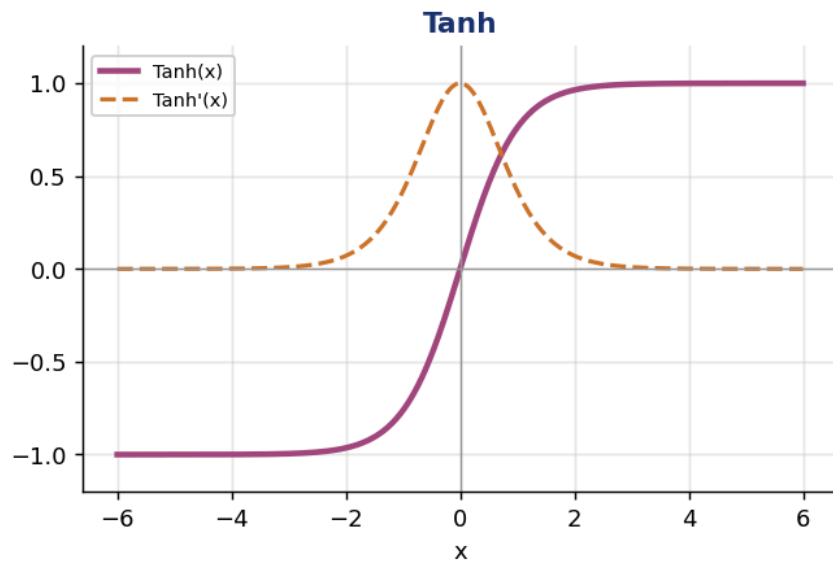
$\sigma(z)=1/(1+e^{-z}) \in (0,1)$. Salida binaria.

Línea punteada: derivada de la función de activación

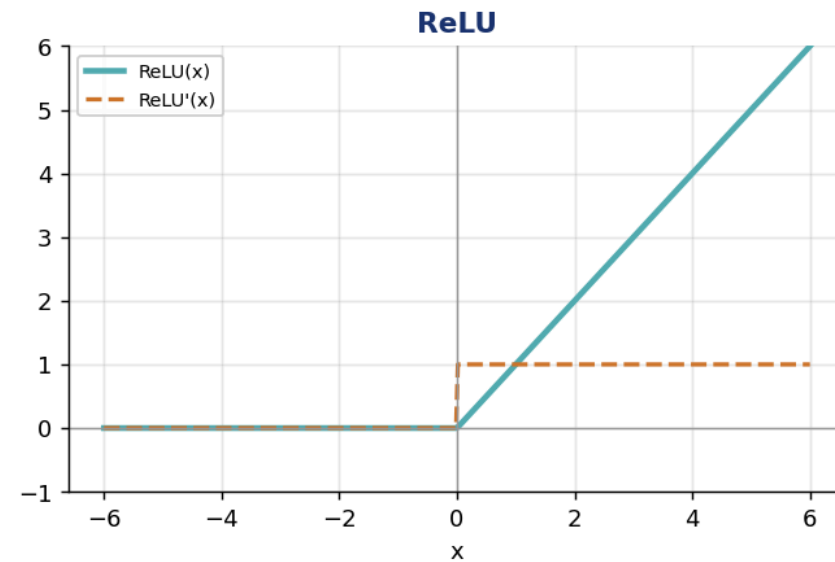
Sigmoide: se **satura** ($\sigma' \leq 0.25$) $\rightarrow$ *vanishing gradient*; salida no centrada en 0.



Tanh y ReLU



$\tanh(z) \in (-1,1)$, centrada en 0; aún satura.

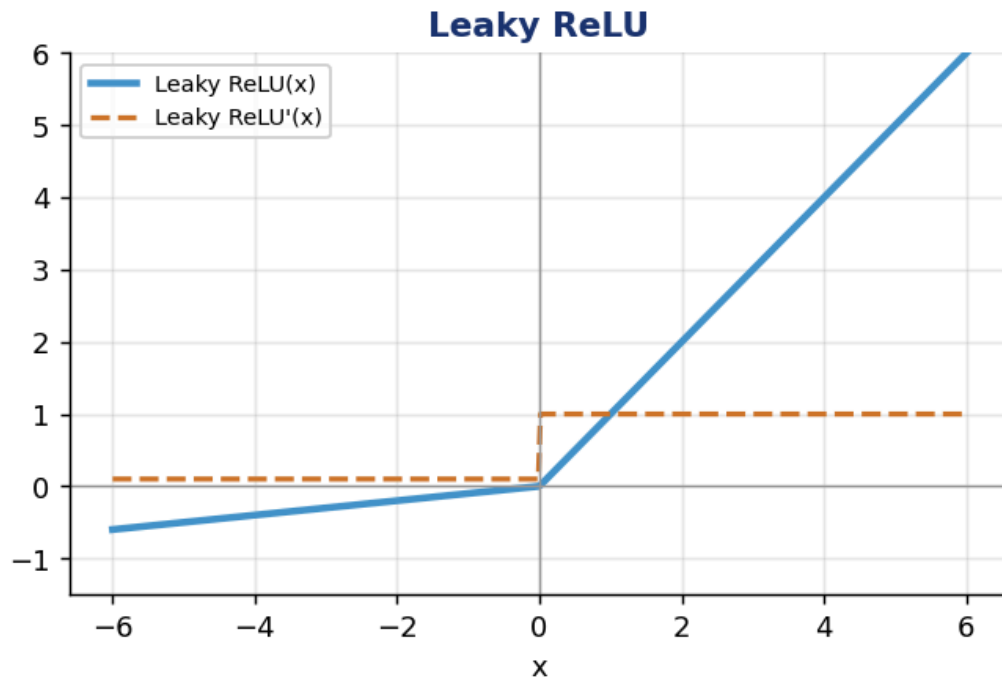


$\text{ReLU}(z) = \max(0, z)$. Derivada 0 o 1.

ReLU es la opción **por defecto** en capas ocultas: derivada **constante** en positivos → evita *vanishing gradient*, muy barata. Riesgo: **neuronas muertas**.



Leaky ReLU

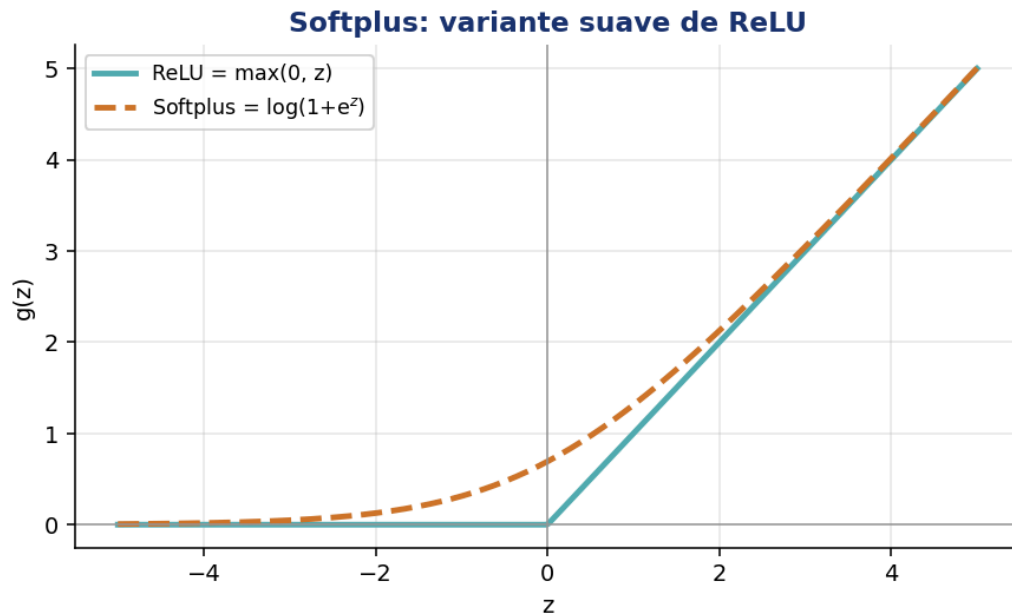


$$\text{LReLU}(z) = \begin{cases} z & z > 0 \\ \alpha z & z \leq 0 \end{cases}$$

Da una pequeña pendiente α a la zona negativa → el gradiente nunca es 0 → **mitiga las neuronas muertas**.



Softplus

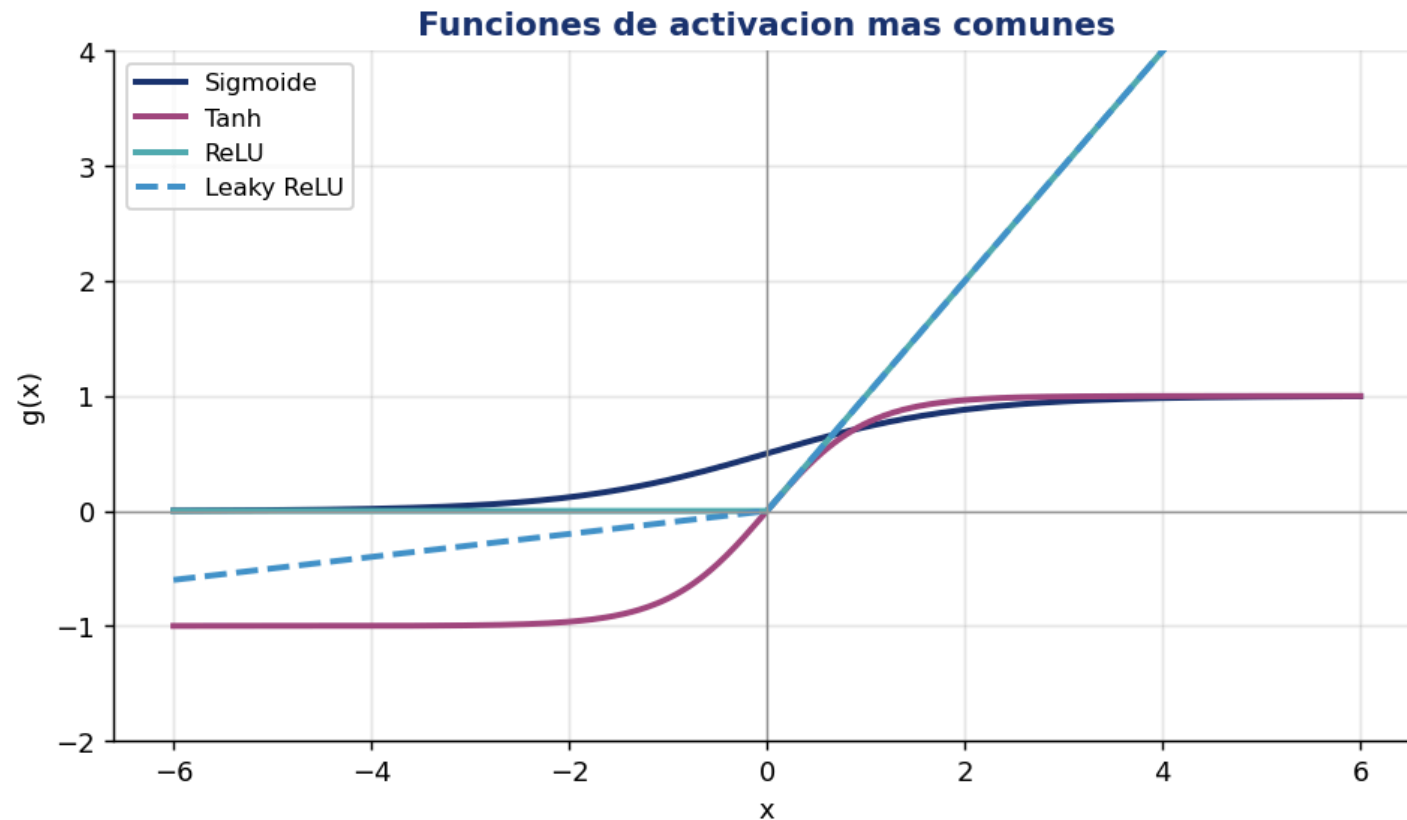


Versión **suave** de ReLU: diferenciable en todo punto.

Útil en la **salida de regresión** cuando se requiere una predicción **siempre positiva** (p. ej. precios).



Comparativa de activaciones



Funciones de activación más comunes. Fuente: elaboración propia (notes.md).



Efectos: tabla resumen

Función	Rango	Derivada	¿Satura?	<i>Vanishing</i>	Uso típico
Identidad	$(-\infty, \infty)$	1	No	—	Salida regresión
Sigmoide	$(0, 1)$	≤ 0.25	Sí	Alto	Salida binaria
Tanh	$(-1, 1)$	≤ 1	Sí	Medio-alto	Ocultas (centrada)
ReLU	$[0, \infty)$	0 o 1	Solo neg.	Bajo	Ocultas (defecto)
Leaky ReLU	$(-\infty, \infty)$	α o 1	No	Bajo	Ocultas
Softmax	$(0, 1)$, suman 1	—	—	—	Salida multiclase

👉 ReLU en ocultas; la salida la dicta la tarea (identidad / sigmoide / softmax).





Ejercicio: funciones de activación

Dado el vector de valores netos $z = [-2, 0, 3]$, calcular **ReLU**, **sigmoide** y **tanh**, y sus **derivadas**. ¿Qué función "satura" en $z = 3$?

Recordatorio: $\sigma'(z) = \sigma(z)(1 - \sigma(z))$, $\tanh'(z) = 1 - \tanh^2(z)$, $\text{ReLU}'(z) = \mathbb{1}[z > 0]$.





Solución: funciones de activación

z	ReLU	$\text{ReLU}'(z)$	$\sigma(z)$	$\tanh(z)$	$\sigma'(z)$	$\tanh'(z)$
-2	0	0	0.119	-0.964	0.105	0.071
0	0	0	0.500	0.000	0.250	1.000
3	3	1	0.953	0.995	0.045	0.010

En $z = 3$, **sigmoide y tanh están casi saturadas**: sus derivadas son diminutas (0.045 y 0.010) $\rightarrow$ poco gradiente. **ReLU mantiene derivada = 1** $\rightarrow$ por eso facilita entrenar redes profundas.



3. Softmax

La salida en clasificación multiclase

¿Para qué sirve softmax?

- Convierte los **logits** (las K salidas lineales, una por clase, $\in \mathbb{R}$) en una **distribución de probabilidad**.

$$\text{softmax}(\mathbf{z})_k = \frac{e^{z_k}}{\sum_{j=1}^K e^{z_j}}$$

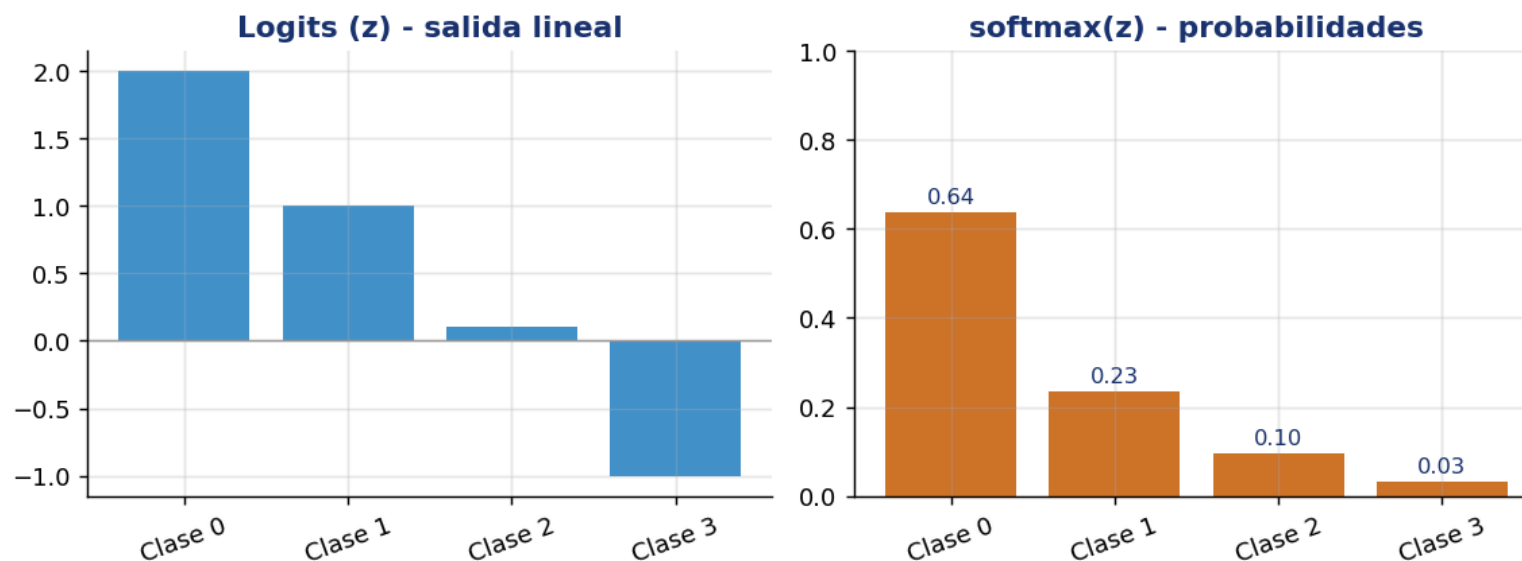
- ✓ Cada salida en $(0, 1)$
- ✓ **Todas suman 1**

- ✓ Monótona
- ✓ La exponencial **amplifica** la clase dominante



Softmax: de logits a probabilidades

Softmax transforma logits en probabilidades (suman 1)



Softmax transforma logits arbitrarios en probabilidades que suman 1. Fuente: elaboración propia (notes.md).



Propiedades clave

Relación con la sigmoide

La sigmoide es softmax con $K = 2$.

- 2 clases $\rightarrow$ sigmoide (1 neurona) o softmax (2).
- $K > 2 \rightarrow$ softmax (K neuronas).

Estabilidad numérica

$$\text{softmax}(\mathbf{z}) = \text{softmax}(\mathbf{z} - c\mathbf{1}).$$

Se resta $c = \max_k z_k$ a cada componente (con $\mathbf{1}$ como vector de unos) para evitar *overflow* en e^{z_k} .

En la práctica (scikit-learn): `MLPClassifier` aplica softmax **automáticamente**.

`predict_proba()` devuelve estas probabilidades; `predict()` la clase de mayor probabilidad.
Se empareja con **entropía cruzada**.





Ejercicio: softmax

Una red de 3 clases produce los logits $\mathbf{z} = [2.0, 1.0, 0.1]$.
Calcular las **probabilidades softmax** y la **clase predicha**.

$$e^2 = 7.389, e^1 = 2.718, e^{0.1} = 1.105.$$





Solución: softmax

$$\sum_j e^{z_j} = 7.389 + 2.718 + 1.105 = 11.212$$

Clase	e^{z_k}	$p_k = e^{z_k} / \sum$
0	7.389	0.659
1	2.718	0.242
2	1.105	0.099

Suman 1.000. **Clase predicha = 0** (mayor probabilidad). Nota: el logit 2.0 solo duplica al 1.0, pero la exponencial le da el **66 %** de la probabilidad.



4. Función de coste

Cuánto se equivoca la red

La función de coste (loss)

- Mide la **distancia** entre lo predicho y lo real. El entrenamiento **minimiza** su promedio.
- Depende de los parámetros: $L(\mathbf{w}, b)$.

Regresión

MSE (media) · MAE (mediana)

Clasificación

Entropía cruzada (log-loss)



Costes de regresión

Error cuadrático medio (MSE)

$$L = \frac{1}{n} \sum_{i=1}^n (\hat{y}^{(i)} - y^{(i)})^2$$

Penaliza fuerte los errores grandes → aprende la **media**.

Error absoluto medio (MAE)

$$L = \frac{1}{n} \sum_{i=1}^n |\hat{y}^{(i)} - y^{(i)}|$$

Robusto a outliers → aprende la **mediana**.



Coste de clasificación: entropía cruzada

$$L_{\text{bin}} = -[y \log p + (1 - y) \log(1 - p)]$$

$$L_{\text{multi}} = -\frac{1}{N} \sum_{i=1}^N \sum_{k=1}^K y_{i,k} \log p_{i,k}$$

Como $y_{i,k} = 1$ solo para la clase correcta, se reduce a $-\log(p_{\text{clase correcta}})$. Minimizarla empuja la probabilidad de la clase correcta hacia 1.





Ejercicio: función de coste

(a) **Clasificación.** La clase verdadera es la **0** y la red predice $p = [0.659, 0.242, 0.099]$ (del ejercicio de softmax). Calcular la **entropía cruzada**.

(b) **Regresión.** Para $y = [3, 5]$ y $\hat{y} = [2.5, 5.5]$, calcular **MSE** y **MAE**.





Solución: función de coste

(a) Entropía cruzada

$$L = -\log(0.659) = \mathbf{0.417}$$

Si predijera 0.9 para la clase correcta:
 $-\log(0.9) = 0.105 \rightarrow$ **menor pérdida**,
mejor.

(b) Regresión

$$\text{MSE} = \frac{0.5^2 + 0.5^2}{2} = \mathbf{0.25}$$

$$\text{MAE} = \frac{|0.5| + |0.5|}{2} = \mathbf{0.5}$$

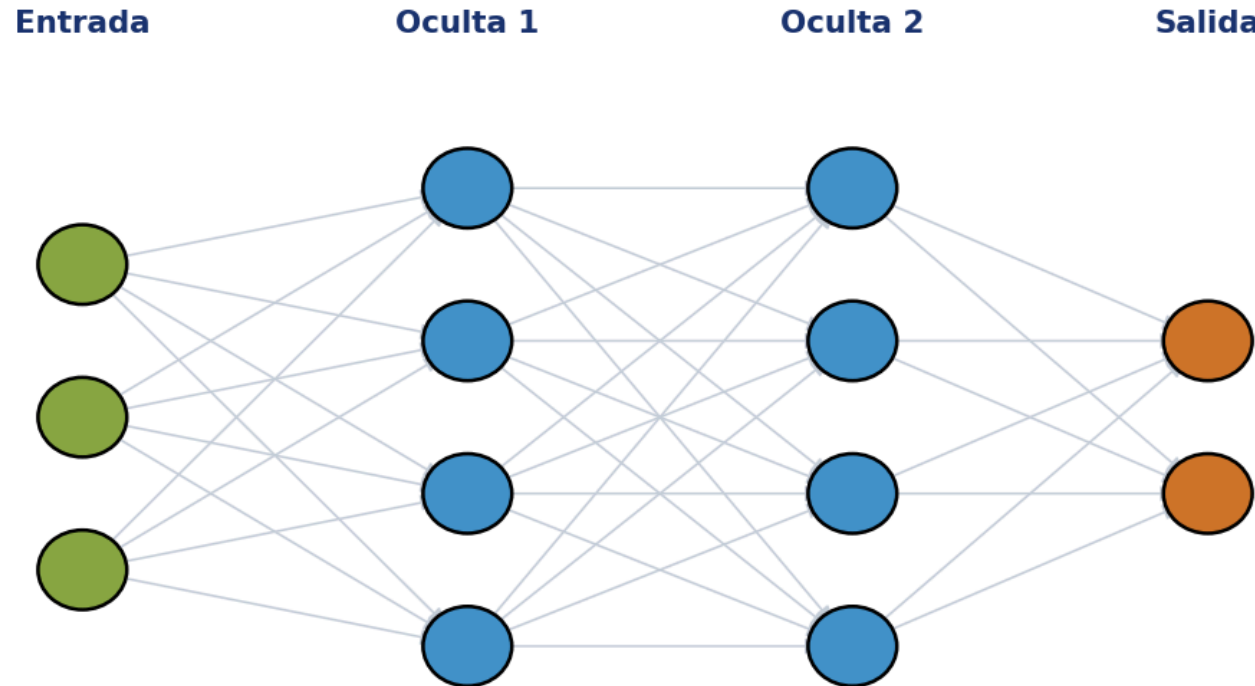


5. El perceptrón multicapa (MLP)

Profundidad y aproximación universal

MLP: capas densas

Perceptron multicapa (MLP): capas densas totalmente conectadas



MLP con dos capas ocultas, totalmente conectado. Fuente: elaboración propia (notes.md).



Aproximador universal

Con **suficientes neuronas**, un MLP puede aproximar **cualquier función continua**.

Profundidad (más capas)

→ representaciones **jerárquicas**.

Anchura (más neuronas)

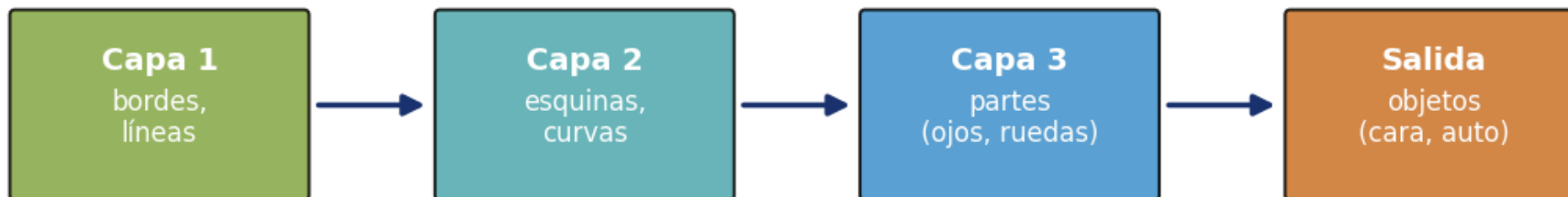
→ más capacidad por nivel.

Ambas aumentan parámetros, tiempo y *riesgo de sobreajuste*.



¿Por qué ayuda la profundidad?

Las capas profundas componen features simples en complejos



Las redes profundas son más **eficientes en parámetros**: las capas aprenden **features jerárquicas** (simples → complejas). Esto habilita el **transfer learning**: reutilizar las capas bajas en tareas nuevas.



6. Entrenamiento

Backpropagation y descenso de gradiente

La idea del entrenamiento

1. Inicializar pesos y bias **aleatorios**.
2. **Forward pass**: propagar y predecir.
3. Calcular la **pérdida**.
4. **Backward pass**: calcular el **gradiente** (responsabilidad de cada parámetro).
5. **Actualizar** parámetros para reducir la pérdida.
6. Repetir 2–5.

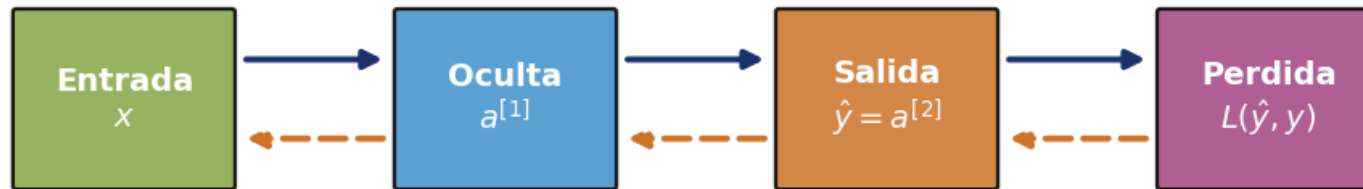
Dos piezas: **backpropagation** (calcular el gradiente) + **descenso de gradiente** (usarlo para actualizar).



El ciclo: forward + backward

Entrenamiento = forward (prediccion) + backward (gradiente) + actualizacion

Forward propagation (calcula la prediccion y la perdida)



Backpropagation (propaga el gradiente $\partial L / \partial \theta$ de la salida hacia la entrada)

Forward propagation calcula la predicción y la pérdida; backpropagation propaga el gradiente de la salida hacia la entrada. Fuente: elaboración propia (notes.md).



Forward propagation

Por capas $l = 1, \dots, L$:

$$z^{[l]} = W^{[l]} a^{[l-1]} + b^{[l]}, \quad a^{[l]} = g^{[l]}(z^{[l]})$$

- $a^{[0]} = \mathbf{x}$ (la entrada).
- La salida es $\hat{y} = a^{[L]}$ (p. ej. probabilidades softmax).
- Con $\hat{y}$ e y se calcula la pérdida L .



Backpropagation: la idea

- La pérdida depende de un peso temprano **solo a través de una cadena** de operaciones.
- La **regla de la cadena** dice: la derivada total es el **producto** de las derivadas de cada eslabón.
- Backprop recorre la red **de la salida a la entrada**, reutilizando cálculos.

Pieza central: el **error de cada capa** $\delta^{[l]} = \frac{\partial L}{\partial z^{[l]}}$ — la "responsabilidad" de esa capa en el error final.



Las 4 ecuaciones de backpropagation

(1) Error de salida: $\delta^{[L]} = \nabla_a L \odot g'^{[L]}(z^{[L]})$

(2) Propagación hacia atrás: $\delta^{[l]} = ((W^{[l+1]})^\top \delta^{[l+1]}) \odot g'^{[l]}(z^{[l]})$

(3) Gradiente de pesos: $\frac{\partial L}{\partial W^{[l]}} = \delta^{[l]} (a^{[l-1]})^\top$

(4) Gradiente de bias: $\frac{\partial L}{\partial b^{[l]}} = \delta^{[l]}$

Con estas 4 ecuaciones se obtiene **todo el gradiente en un solo recorrido** salida → entrada.



Dos consecuencias importantes

Softmax + entropía cruzada

El error de salida se simplifica a:

$$\delta^{[L]} = \hat{y} - y$$

¡La derivada de softmax se cancela! Por eso van **siempre juntas**.

Vanishing gradient

En la ec. (2), si g' es pequeña (sigmoide saturada), $\delta^{[l]}$ se **encoge** capa tras capa. ReLU ($g' = 1$) no lo atenúa.





Ejercicio: backpropagation

Red mínima: 1 entrada $\rightarrow$ 1 neurona oculta (**sigmoide**) $\rightarrow$ 1 salida (**identidad**), pérdida $\text{MSE} = \frac{1}{2}(\hat{y} - y)^2$.

Parámetros: $x = 1$, $w_1 = 0.5$, $b_1 = 0$, $w_2 = 2$, $b_2 = 0$; objetivo $y = 1$.

Calcular el forward pass, los gradientes de los 4 parámetros, y actualizarlos con $\eta = 0.1$.





Solución: forward

$$z_1 = w_1x + b_1 = 0.5 \cdot 1 + 0 = 0.5$$

$$a_1 = \sigma(0.5) = 0.622$$

$$z_2 = w_2a_1 + b_2 = 2 \cdot 0.622 = 1.244$$

$$\hat{y} = z_2 = 1.244 \quad (\text{salida identidad})$$

$$L = \frac{1}{2}(1.244 - 1)^2 = \frac{1}{2}(0.244)^2 = \mathbf{0.0298}$$





Solución: backward + update

Backward

$$\delta_2 = (\hat{y} - y) \cdot 1 = 0.244$$

$$\frac{\partial L}{\partial w_2} = \delta_2 a_1 = 0.152$$

$$\frac{\partial L}{\partial b_2} = 0.244$$

$$\delta_1 = w_2 \delta_2 \cdot \sigma'(z_1) = 2(0.244)(0.235) = 0.115$$

$$\frac{\partial L}{\partial w_1} = \delta_1 x = 0.115, \quad \frac{\partial L}{\partial b_1} = 0.115$$

Actualización ($\eta = 0.1$)

$$w_2 \leftarrow 2 - 0.1(0.152) = 1.985$$

$$b_2 \leftarrow 0 - 0.1(0.244) = -0.024$$

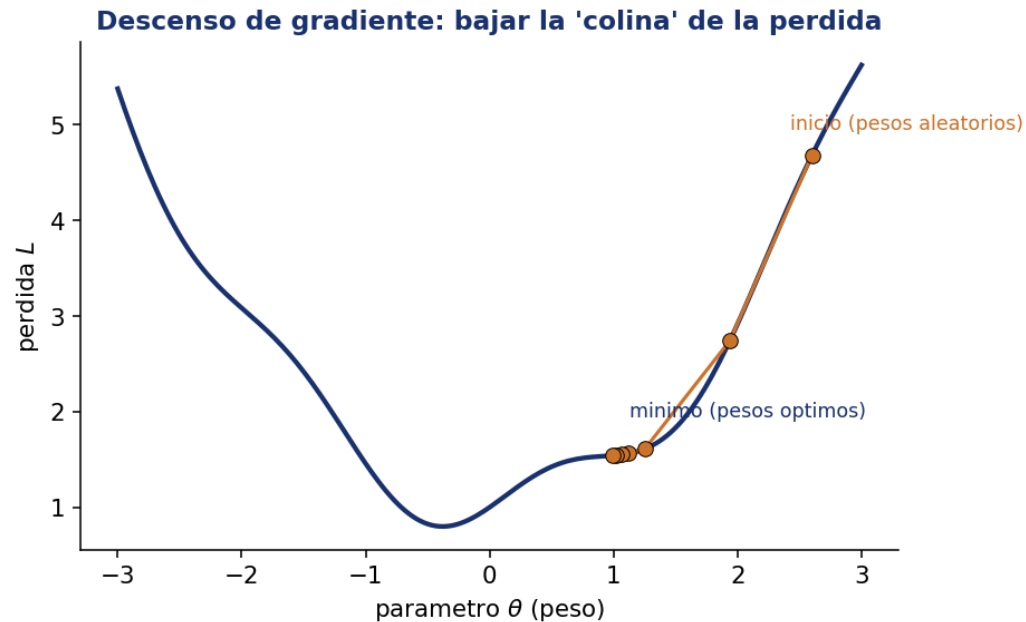
$$w_1 \leftarrow 0.5 - 0.1(0.115) = 0.489$$

$$b_1 \leftarrow 0 - 0.1(0.115) = -0.011$$

Los pesos se mueven en la dirección que **reduce** la pérdida. Repetir miles de veces = entrenar.



Descenso de gradiente



$$\theta \leftarrow \theta - \eta \frac{\partial L}{\partial \theta}$$

Damos pasos en la dirección **opuesta** al gradiente: bajar la "colina" de la pérdida hasta un valle.
 η = **learning rate**.

Pasos sucesivos del descenso de gradiente. Fuente: elaboración propia (notes.md).



Batch, SGD y mini-batch

Batch

Todas las observaciones por paso. Exacto pero **costoso**.

SGD

Una observación por paso. Rápido y ruidoso; escapa mínimos locales.

Mini-batch

Lotes de 32–256. **Lo habitual**: equilibrio y vectorización.

Época = una pasada completa por todo el conjunto de entrenamiento.



Optimizadores adaptativos

- **Momentum**: acumula "velocidad" → acelera direcciones consistentes, amortigua oscilaciones.
- **RMSprop**: escala el paso según la magnitud reciente del gradiente.
- **Adam**: combina momentum + RMSprop. **Por defecto** (`solver='adam'`).

Conjuntos **grandes** → `adam`. Conjuntos **pequeños** → `lbfgs` (segundo orden, converge rápido). La **inicialización** (Glorot/He) también importa para que fluya el gradiente.



El problema del *vanishing gradient*

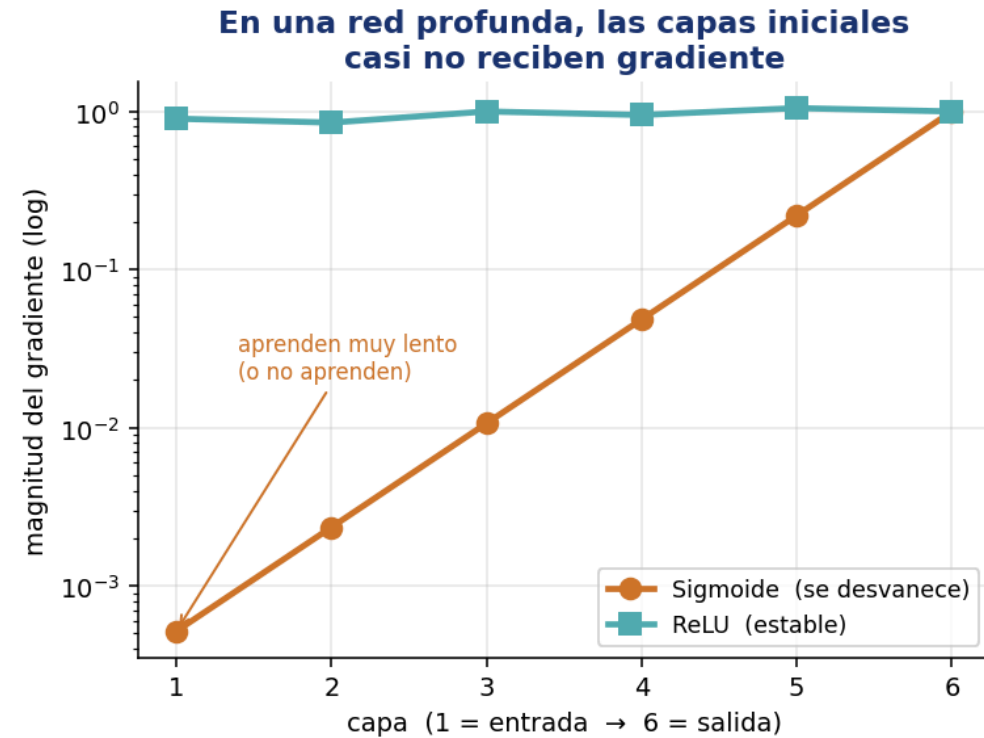
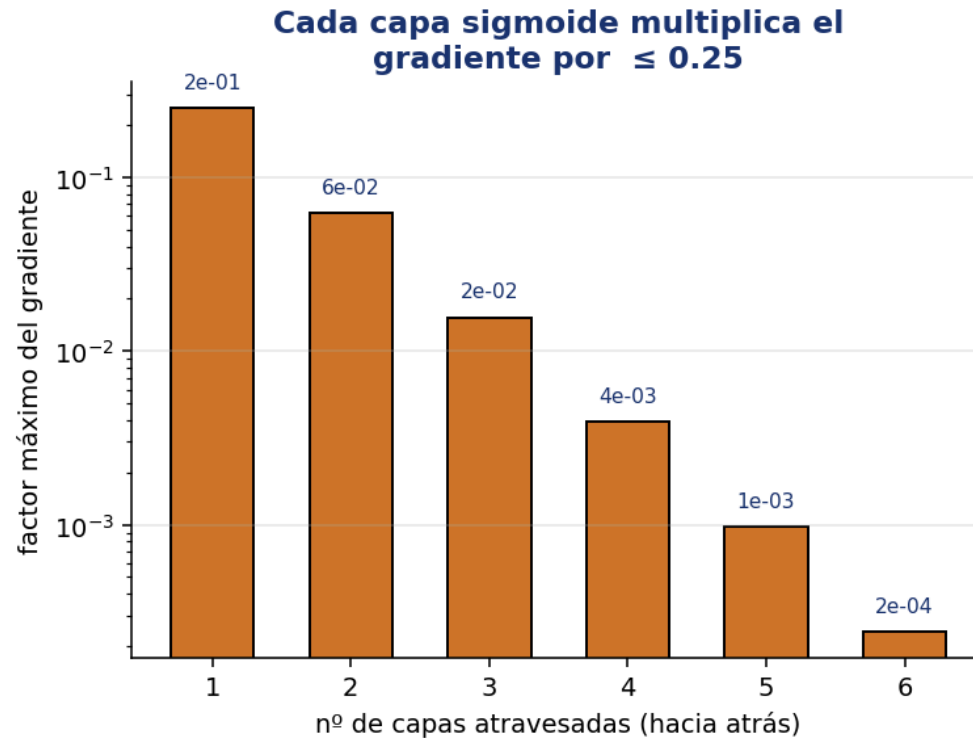
Para llegar a las primeras capas, el gradiente atraviesa **muchas** capas y en cada una se multiplica por $g'(z)$.

$$\delta^{[l]} = ((W^{[l+1]})^\top \delta^{[l+1]}) \odot g'(z^{[l]})$$

Con **sigmoide**, $\sigma'(z) \leq 0.25$. Multiplicar muchos factores pequeños $\rightarrow 0.25^L \rightarrow 0$: el gradiente se **desvanece** y las capas iniciales **casi no aprenden**.



Vanishing gradient: visto en números



Izq.: cada capa sigmoide encoge el gradiente (≤ 0.25). Der.: en una red profunda, las capas cercanas a la entrada casi no reciben gradiente (ReLU lo mantiene estable). Fuente: elaboración propia.



¿Cómo se mitiga?

ReLU (y variantes)

Derivada = 1 en positivos → no atenúa el gradiente.
La solución más simple.

Batch normalization

Normaliza activaciones dentro de la red para que no saturen.

Inicialización (Glorot/He)

Mantiene la escala de activaciones y gradientes entre capas.

Conexiones residuales (ResNet)

"Atajos" por donde el gradiente fluye sin atenuarse
→ redes de cientos de capas.

 Muchas innovaciones del deep learning moderno existen para que el ****gradiente siga fluyendo****.



7. Preprocesado

Antes de entrenar

Dos transformaciones mínimas

One-hot encoding

Cada nivel de una categórica → una columna *dummy* $\{0, 1\}$.

`color` → `color_rojo`, `color_verde`,
`color_azul`.

Estandarización

Igualar escalas de las numéricas.

- **Z-score** (`StandardScaler`): media 0, desv. 1.
- **Min-Max** (`MinMaxScaler`): rango $[0, 1]$.

⚠ Nunca estandarizar variables **después** de binarizarlas. El escalado va solo a las numéricas.



8. Hiperparámetros

Configurar para no sobreajustar

Hiperparámetros clave

- **Nº y tamaño de capas** (`hidden_layer_sizes`): capacidad del modelo.
- **Activación** (`activation`): ReLU por defecto en ocultas.
- **Learning rate** (`learning_rate_init`): alto → inestable; bajo → lento.
- **Solver** (`solver`): `adam`, `lbfgs`, `sgd`.
- **Regularización** (`alpha`, L2): penaliza pesos grandes → suaviza fronteras.
- **Early stopping**: parar cuando la validación deja de mejorar.



Regularización contra el sobreajuste

L1 / L2 (weight decay)

Penalizan pesos grandes. Evitan que pocas neuronas dominen; el ruido $\rightarrow$ pesos ≈ 0 .

Dropout

Desactiva aleatoriamente neuronas (0.2–0.5) en cada iteración. Estándar en deep learning (Keras/PyTorch).

La **flexibilidad** de las redes es un arma de doble filo: aprenden mucho pero **sobreajustan**. Los hiperparámetros marcan la diferencia.



Guías prácticas de ajuste

Capas / neuronas

Empezar con 1–2 capas. Mismo nº de neuronas por capa. "Pantalones elásticos": sobredimensionar + *early stopping*.

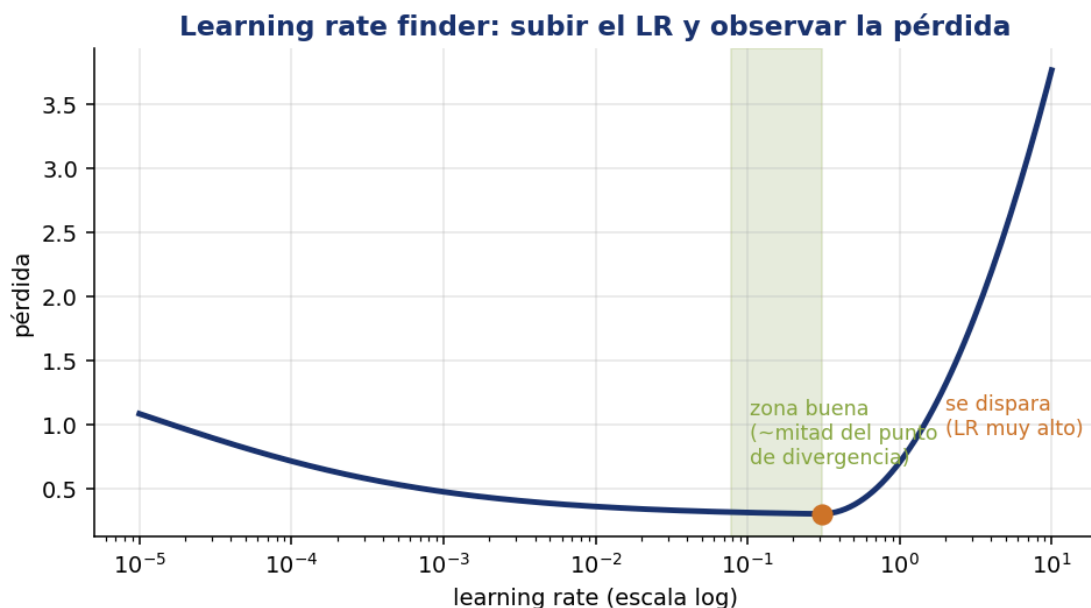
Batch / optimizador

Lotes pequeños (2–32) generalizan mejor. **Adam** + **ReLU** buenos por defecto.

💡 El learning rate óptimo ****depende**** del resto (sobre todo del batch size).



Learning rate finder



Subir el LR exponencialmente y graficar la pérdida.

El LR óptimo está **un poco por debajo** del punto donde la pérdida se dispara ($\approx$ la mitad del LR que diverge).

Learning rate finder. Fuente: elaboración propia (inspirado en Géron, 2023, cap. 9).



Arquitecturas típicas: regresión

Hiperparámetro	Valor típico
Nº de capas ocultas	1 a 5
Nº de neuronas por capa	10 a 100
Neuronas de salida	1 por dimensión
Activación oculta	ReLU
Activación de salida	Ninguna · ReLU/softplus (positiva) · sigmoide/tanh (acotada)



Arquitecturas típicas: clasificación

Hiperparámetro	Binaria	Multietiqueta	Multiclase
Neuronas de salida	1	1 por etiqueta	1 por clase
Activación de salida	Sigmoide	Sigmoide	Softmax
Función de coste	Entropía cruzada	Entropía cruzada	Entropía cruzada

⚠ Si usas softmax, las redes son **sobreentrenadas**: `predict_proba` puede dar 0.99 a una predicción errónea. Tomar las probabilidades con pinzas.



9. ANN en scikit-learn

Efectos prácticos y optimización

MLPClassifier / MLPRegressor

```
from sklearn.neural_network import MLPClassifier
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler

modelo = Pipeline([
    ("scaler", StandardScaler()),
    ("mlp", MLPClassifier(hidden_layer_sizes=(20, 20),
                          activation="relu", solver="adam",
                          alpha=1e-4, max_iter=1000,
                          random_state=42)),
])
modelo.fit(X_train, y_train)
modelo.predict_proba(X_test[:1]) # probabilidades softmax
```

El notebook `efectos_hiperparametros.ipynb` explora estos efectos sobre datos 2D.



Efecto: n° de capas y neuronas

Datos con **solapamiento** (no separables) — para que el efecto se note.

- (5,), (10,) → fronteras suaves, generalizan bien.
- (50, 50, 50) → `acc_train = 1.0` pero fronteras "isla" = **sobreajuste**.
 -  Más capacidad ****no**** significa mejor generalización.



Efecto: función de activación

- `identity` → frontera **lineal** (¡aunque haya 2 capas!).
- `tanh` / `logistic` → fronteras suaves y curvas.
- `relu` → fronteras lineales **a trozos**.

Confirma la teoría: **sin no linealidad, no hay fronteras curvas.**



Efecto: learning rate y regularización

Learning rate

Existe una **zona óptima**: muy bajo → no aprende;
muy alto → inestable, accuracy cae.

Regularización α

A mayor α → fronteras **más suaves**. En exceso
→ *underfitting*.

Como los hiperparámetros **interactúan**, se ajustan **en conjunto** por validación cruzada.



Optimización con GridSearchCV

```
from sklearn.model_selection import GridSearchCV

param_grid = {
    "mlp__hidden_layer_sizes": [(10,), (20,), (20, 20), (50, 50)],
    "mlp__activation": ["tanh", "relu"],
    "mlp__alpha": [1e-4, 1e-2, 1e-1],
}
grid = GridSearchCV(pipe, param_grid, scoring="accuracy", cv=5, n_jobs=-1)
grid.fit(X, y)
grid.best_params_    # mejor combinación
```

Prueba **todas** las combinaciones. Exhaustivo pero el coste **crece exponencialmente** con el nº de hiperparámetros.



Optimización con Optuna

Explora el espacio de forma **inteligente** (muestreador bayesiano **TPE**): aprende de los intentos previos y concentra la búsqueda.

```
import optuna

def objective(trial):
    n_layers = trial.suggest_int("n_layers", 1, 3)
    n_units  = trial.suggest_int("n_units", 5, 60)
    alpha    = trial.suggest_float("alpha", 1e-5, 1e0, log=True)
    ...
    return cross_val_score(modelo, X, y, cv=5).mean()

study = optuna.create_study(direction="maximize")
study.optimize(objective, n_trials=40)
```



GridSearchCV vs Optuna

	GridSearchCV	Optuna (TPE)
Estrategia	Todas las combinaciones	Bayesiana, adaptativa
Espacio	Discreto (grilla fija)	Continuo + discreto
Evaluaciones	Crecen exponencialmente	Pocas, dirigidas
Extras	—	Historia, importancia de hiperparámetros

En el notebook, Optuna alcanza una accuracy ligeramente superior con una red **más simple** — y permite rangos **continuos** de `alpha`.

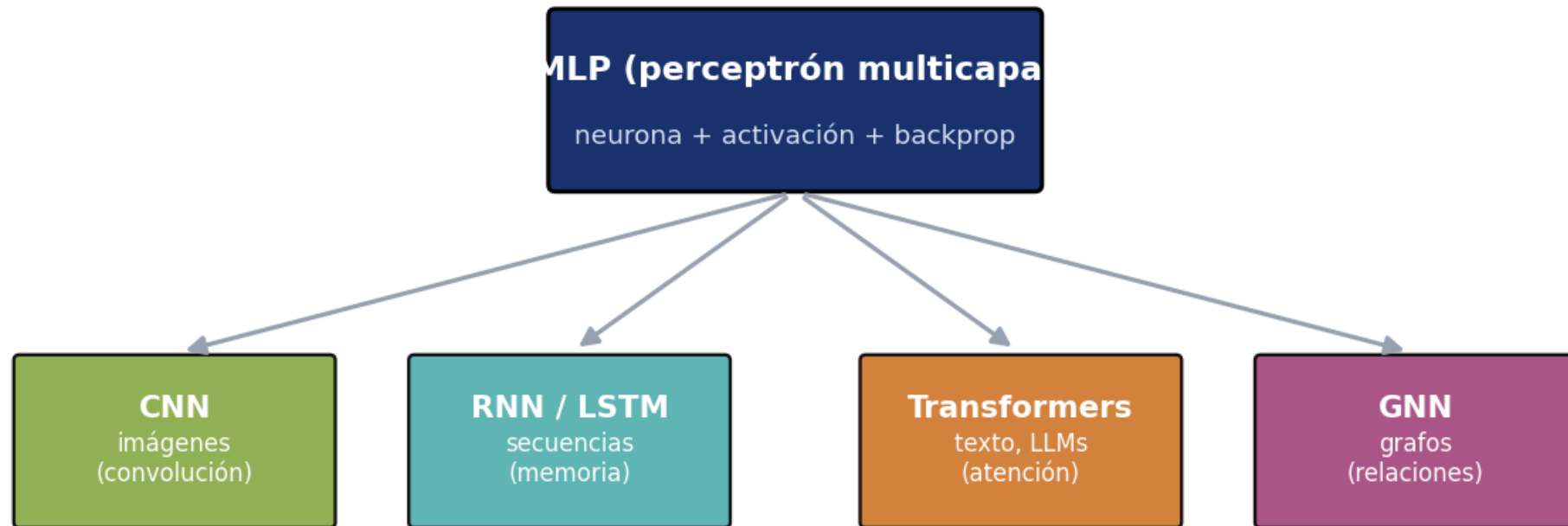


10. Más allá del MLP

La base del deep learning moderno

El MLP es la base de todo

El MLP es la base arquitectónica del deep learning moderno



Todas heredan la misma idea: capas de unidades entrenadas con descenso de gradiente y backpropagation

Las arquitecturas modernas especializan la misma idea: neurona + activación + backprop. Fuente: elaboración propia.



¿Cuándo usar una ANN?

Tipo de dato	Mejor opción habitual
Tabular (filas/columnas)	A menudo gradient boosting (XGBoost, LightGBM); el MLP rara vez gana
Imágenes	CNN (o Transformers de visión)
Texto / lenguaje	Transformers (LLMs)
Secuencias / series	RNN/LSTM o Transformers
Grafos	GNN

🧭 No estudiamos el MLP por ser el mejor datos tabulares, sino porque es la **pieza fundacional**: quien entiende cómo se entrena un MLP entiende CNN, Transformers y todo el *deep learning*.



11. Recap

Lo que aprendimos

Recap (1/3): estructura y activaciones

- Una **red** = capas de **neuronas**; cada una hace **suma ponderada + activación**.
- La **activación no lineal** es lo que da poder: sin ella, todo colapsa a un modelo lineal.
- **ReLU** en ocultas (evita *vanishing gradient*); la **salida** la dicta la tarea.
- **Softmax** en la salida multiclase: logits → **distribución de probabilidad** que suma 1, emparejada con **entropía cruzada**.



Recap (2/3): entrenamiento

- **Coste**: MSE/MAE (regresión); **entropía cruzada** (clasificación).
- **Forward** calcula la predicción y la pérdida.
- **Backpropagation** = regla de la cadena salida $\rightarrow$ entrada; las **4 ecuaciones** dan todo el gradiente.
- Con softmax + entropía cruzada: $\delta^{[L]} = \hat{y} - y$.
- **Descenso de gradiente** (batch / SGD / mini-batch) + optimizadores (**Adam**).



Recap (3/3): práctica

- **Preprocesar**: one-hot + estandarización (¡no escalar dummies!).
- **Hiperparámetros**: capas/neuronas, learning rate, `alpha`, solver, early stopping.
- Más capacidad **no** = mejor: cuidado con el **sobreajuste**.
- Ajustar **en conjunto** con validación cruzada: **GridSearchCV** (exhaustivo) → **Optuna** (inteligente).



¿Preguntas?

Redes Neuronales Artificiales — Deep Learning, UCU

Material: `notes.md` · `efectos_hiperparametros.ipynb`